

Toolformer: 语言模型可以自我学习使用工具

Timo Schick Jane Dwivedi-Yu Roberto Dessì† Roberta Raileanu
Maria Lomeli Luke Zettlemoyer Nicola Cancedda Thomas Scialom

Meta AI 研究 † 波佩鲁法大学

摘要

语言模型 (LMs) 展现出在规模化条件下，仅通过少量示例或文本指令即可解决新任务的卓越能力。它们也存在矛盾地在基本功能方面表现不佳，例如算术或事实查询，而这些任务由更简单、更小的模型表现得更好。在本文中，我们展示了 LMs 可以通过简单的 API 自我学习使用外部工具，实现两者的最佳结合。我们引入了 *Toolformer*，一种训练用以决定调用哪些 API、何时调用、传递何种参数，以及如何将结果最佳融入未来的 Token 预测中的模型。这一过程采用自监督方式完成，只需为每个 API 提供少量示范。我们整合了多种工具，包括计算器、问答系统、搜索引擎、翻译系统和日历。*Toolformer* 在多种下游任务中实现了显著提升的零-shot 性能，常常与更大规模的模型竞争，同时不牺牲其核心的语言建模能力。

1 引言

大型语言模型在多种自然语言处理任务中实现了令人印象深刻的零-shot 和少-shot 结果 (Brown 等人, 2020; Chowdhery 等人, 2022, 等)，并展现出若干新兴能力 (Wei 等人, 2022)。然而，所有这些模型都存在一些固有的局限性，最多只能通过进一步扩展规模部分缓解。这些局限性包括无法获取关于近期事件的最新信息 (Komeili 等人, 2022) 以及相关的事实幻觉倾向 (Maynez 等人, 2020; Ji 等人, 2022)，在理解低资源语言方面存在困难 (Lin 等人, 2021)，缺乏进行精确计算的数学能力 (Patel 等人, 2021)，以及对时间流逝缺乏意识 (Dhingra 等人, 2022)。

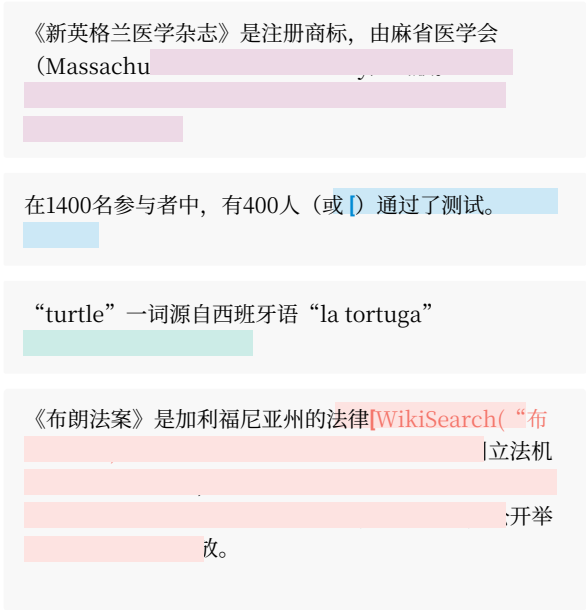


图1: Toolformer的示例预测。模型自主决定调用不同的API (从上到下: 问答系统、计算器、机器翻译系统和维基百科搜索引擎) 以获取完成文本所需的信息。

克服当今语言模型这些限制的一个简单方法是赋予它们使用外部工具的能力，例如搜索引擎、计算器或日历。然而，现有的方法要么依赖大量的人类标注 (Komeili 等人, 2022; Thoppilan 等人, 2022) 要么将工具的使用限制在特定任务的设置中 (例如, Gao 等人, 2022; Parisi 等人, 2022)，阻碍了工具在语言模型中的更广泛应用。因此，我们提出 *Toolformer*，一种以新颖方式学习使用工具的模型，满足以下期望：

工具的使用应通过自我监督的方式学习，而无需大量的人工标注。这是一种重要的

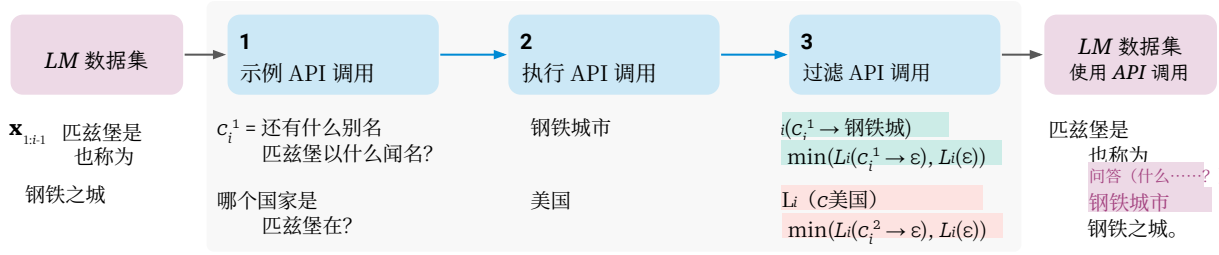


图2: 我们的方法的关键步骤, 以问答工具为例进行说明: 给定输入文本 *question answering*, 我们首先采样一个位置 i 及对应的 API 调用候选项 $c_i^1, c_i^2, \dots, c_i^k$ 。然后执行这些 API 调用, 并筛选出所有不能在后续词中降低损失 L_i 的调用。所有剩余的 API 调用与原始文本交织, 形成新的文本 $\mathbf{x}^*$ 。

不仅仅因为与此类注释相关的成本, 还因为人类认为有用的内容可能与模型认为有用的内容不同。

语言模型不应失去其通用性, 并且应能够自主决定何时以及如何使用哪种工具。与现有方法相比, 这使得工具的使用更加全面, 不局限于特定任务。

我们的实现这些目标的方法基于最近提出的利用大型语言模型进行<in- context learning> (Brown 等人, 2020) 从头生成完整数据集的思想 (Schick 和 Schütze, 2021b; Honovich 等人, 2022; Wang 等人, 2022): 只需提供少量人工编写的API使用示例, 我们让语言模型对大量语言建模数据进行注释, 标注潜在的API调用。然后, 我们使用自监督损失来判断哪些API调用实际上有助于模型预测未来的标记。最后, 我们在模型认为有用的API调用上对其进行微调。如图1所示, 通过这种简单的方法, 语言模型可以学会控制各种工具, 并自主选择何时以及如何使用这些工具。

作为我们的方法对所使用的数据集保持无关性, 我们可以将其应用于最初用于预训练模型的完全相同的数据集。这确保了模型不会失去其通用性和语言建模能力。我们在多种不同的下游任务上进行了实验, 证明了在学习使用工具后, 基于预训练GPT-J模型 (Wang 和 Komatsuzaki, 2021, 参数为6.7B) 的 Toolformer 在零样本任务中表现出更强的性能, 明显优于规模更大的GPT-3模型 (Brown 等, 2020) 及其。

在各种任务上的其他几个基线。

2 方法

我们的目标是使语言模型 M 具备通过API调用使用不同工具的能力。我们要求每个API的输入和输出都能表示为文本序列。这允许在任何给定文本中无缝插入API调用, 使用特殊标记标识每次调用的起始和结束。

我们将每次 API 调用表示为一个元组 $c = (a_c, i_c)$, 其中 a_c 是 API 的名称, i_c 是对应的输入。给定一个 API 调用 c 及其对应的结果 r , 我们用线性化序列分别表示不包含和包含其结果的 API 调用:

$$\begin{aligned} e(c) &= \langle \text{API} \rangle a_c (i_c) \langle / \text{API} \rangle \\ e(c, r) &= \langle \text{API} \rangle a_c (i_c) \rightarrow r \langle / \text{API} \rangle \end{aligned}$$

where “ $\langle \text{API} \rangle$ ”, “ $\langle / \text{API} \rangle$ ” 和 “ $\rightarrow$ ” 是特殊标记。¹ 一些插入到文本序列中的线性化API调用示例显示在图1中。

给定一个数据集 $\mathcal{C} = \{\mathbf{x}^1, \dots, \mathbf{x}^{|\mathcal{C}|}\}$ 由纯文本组成的数据集, 我们首先将其转换为一个 $\mathcal{C}^*$ 通过API调用增强的数据集。这个过程分为三个步骤, 如图2所示: 首先, 我们利用 M 的上下文学习能力, 采样大量潜在的API调用。然后, 我们执行这些API调用, 最后检查获得的响应是否有助于预测未来的标记; 这作为筛选标准。在筛选后, 我们将不同工具的API调用合并, 得到增强的数据集 $\mathcal{C}^*$, 并进行微调

¹ 在实践中, 我们使用标记序列 “ $\langle$ ”、“ $\rangle$ ” 和 “ $\rightarrow$ ” 分别表示 “ $\langle \text{API} \rangle$ ”、“ $\langle / \text{API} \rangle$ ” 和 “ $\rightarrow$ ”。这使得我们的方法无需修改现有语言模型的词汇表即可实现。出于可读性考虑, 在本节中我们仍然将它们称为 “ $\langle \text{API} \rangle$ ”、“ $\langle / \text{API} \rangle$ ” 和 “ $\rightarrow$ ”。

您的任务是向一段文本中添加问答API调用。这些问题应帮助您获取完成文本所需的信息。您可以通过写作“[QA(question)]”来调用API，其中“question”是您想要提问的问题。以下是一些API调用的示例：

乔·拜登出生于宾夕法尼亚州斯克兰顿。

乔·拜登出生于斯克兰顿，宾夕法尼亚州。

可口可乐，或称可乐，是由可口可乐公司生产的一种碳酸软饮料。

Output: 可口可乐，或 []，Coke，是由可口可乐公司生

x

Toolformer: 语言模型可以自我学习使用工具2 方法

图 3：用于生成问答工具 API 调用的示例提示 $P(\mathbf{x})$

在该数据集上自身进行。每个步骤的详细描述如下。

采样API调用对于每个API，我们编写一个提示 $P(\mathbf{x})$ ，鼓励语言模型对示例 $\mathbf{x} = x_1, \dots, x_n$ ，通过API调用进行注释。针对问答工具的此类提示示例如图3所示；所有使用的提示在附录A.2中展示。设 $p_M(z_{n+1} | z_1, \dots, z_n)$ 为 M 赋予的概率，用于序列 z_{n+1} 作为后续的标记 $z_1, \dots, z_n$ 。我们首先通过计算，为每个 $i \in \{1, \dots, n\}$ ，概率

$$p_i = p_M(\langle \text{API} \rangle | P(\mathbf{x}), x_{1:i-1})$$

that M 将API调用的起始位置分配给位置 i 。给定采样阈值 τ_s ，我们保留所有位置 $I = \{i | p_i > \tau_s\}$ ；如果这些位置超过 k 个，则只保留前 k 个。

对于每个位置 $i \in I$ ，我们随后通过从给定序列 $[P(\mathbf{x})]$ 采样，获得最多 m 次API调用 $c_i^1, \dots, c_i^m$ ，以序列 M 为条件 $x_1, \dots, x_{i-1}$ ， $\langle \text{API} \rangle$ 作为前缀， $\langle / \text{API} \rangle$ 作为序列结束标记。²

² 我们丢弃所有 M 未生成 $\langle / \text{API} \rangle$ 令牌的示例。

执行 API 调用作为下一步，我们执行由 M 生成的所有 API 调用以获取相应的结果。具体的执行方式完全取决于 API 本身——例如，可能涉及调用另一个神经网络、执行 Python 脚本或使用检索系统在大型语料库中进行搜索。每个 API 调用 c_i 的响应都需要是一个单一的文本序列 r_{i0} 。

过滤API调用令 i 为API调用 c_i 在序列 $\mathbf{x} = x_1, \dots, x_n$ 中的位置，并令 r_i 为API的响应。此外，给定一组序列 $(w_i | i \in \mathbb{N})$ 的权重，让

$$L_i(\mathbf{z}) = - \sum_{j=i}^n w_{j-i} \cdot \log p_M(x_j | \mathbf{z}, x_{1:j-1})$$

如果模型以 $\mathbf{z}$ 作为前缀，则 M 在标记 $x_i, \dots, x_n$ 上的加权交叉熵损失。我们比较该损失的两种不同实例：

$$\begin{aligned} L_i^+ &= L_i(\mathbf{e}(c_i, r_i)) \\ L_i^- &= \min(L_i(\varepsilon), L_i(\mathbf{e}(c_i, \varepsilon))) \end{aligned}$$

其中 ε 表示空序列。前者是在所有标记 $x_i, \dots, x_n$ 上的加权损失，如果 API 调用及其结果作为前缀提供给 M ；后者是在 (i) 完全不进行 API 调用和 (ii) 进行 API 调用但不提供响应所获得的最小损失。从直观上看，API 调用对于 M 是有帮助的，如果提供该调用的输入和输出使模型更容易预测未来的标记，相较于完全不接收 API 调用或仅接收其输入。给定过滤阈值 τ_f ，因此我们只保留满足条件的 API 调用

$$L_i^- - L_i^+ \geq \tau_f$$

成立，即添加API调用及其结果 减少损失至少 τ_f ，与不进行任何API调用或未获得任何结果相比。

模型微调在对所有API的调用进行采样和筛选后，我们最终将剩余的API调用合并，并将它们与原始输入交错。也就是说，对于一个输入文本 $\mathbf{x} = x_1, \dots, x_n$ ，以及在位置 i 的对应API调用和结果 (c_i, r_i) ，我们构建新的序列 $\mathbf{x}^* =$

我们提供 $\mathbf{e}(c_i, r_i)$ 作为前缀，而不是插入到位置 i ，因为 M 尚未在包含 API 调用的任何示例上进行微调，因此将其插入到 $\mathbf{x}$ 中间会中断流程，并且不符合预训练语料中的模式，从而降低困惑度。

$x_{1:i-1}, e(c_i, r_i) \ x_{i:n}$ ；我们对具有多个API调用的文本也采用类似的方法。对所有 $\mathbf{x} \in \mathcal{C}$ 结果进行处理，得到新的数据集 $\mathcal{C}^*$ ，该数据集增加了API调用。我们使用这个新数据集对 M 进行微调，采用标准的语言模型训练目标。关键的是，除了插入的API调用外，增强的数据集 $\mathcal{C}^*$ 与 $\mathcal{C}$ （原始数据集）包含完全相同的文本。因此，在 $\mathcal{C}^*$ 上微调 M ，使其暴露于与在 $\mathcal{C}$ 上微调相同的内容。此外，由于API调用被插入在那些有助于 M 预测未来标记的确切位置和输入中，基于 $\mathcal{C}^*$ 的微调使语言模型能够仅凭自身反馈，决定何时以及如何使用哪种工具。

推理在使用我们的方法对 M 进行微调后生成文本时，我们会进行常规解码，直到 M 产生“→”标记，表明它接下来预期API调用的响应。此时，我们中断解码过程，调用相应的API以获取响应，并在插入响应和`</API>`标记后，继续解码过程。

3 工具

我们探索了多种工具，以应对常规语言模型的不同不足之处。我们对这些工具唯一的限制是：(i) 它们的输入和输出都可以用文本序列表示，以及 (ii) 我们可以获得一些关于其预期用途的示范。具体而言，我们探索了以下五种工具：问答系统、维基百科搜索引擎、计算器、日历和机器翻译系统。每种工具的潜在调用示例及其对应的返回字符串如表所示。我们将在下文简要讨论所有工具；更多细节可参见附录。

Question Answering 我们的第一个工具是基于另一种语言模型的问答系统，能够回答简单的事实性问题。具体而言，我们使用`Atlas`（Izacard 等人，2022），一种在自然问答（Kwiatkowski 等人，2019）上微调的检索增强型语言模型。

Calculator 作为第二个工具，我们使用一个可以执行简单数值计算的计算器；我们仅支持四则运算。结果始终保留两位小数。

维基百科搜索我们的第三个工具是一个搜索引擎，给定搜索词后，返回简短文本

Wikipedia的片段。与我们的问答工具相比，这种搜索使模型能够获取关于某一主题的更全面的信息，但需要它自行提取相关部分。作为我们的搜索引擎，我们使用BM25检索器（Robertson等人，1995；Baeza-Yates等人，1999），该检索器对KILT的Wikipedia数据集进行索引（Petroni等人，2021）。

机器翻译系统我们的第四个工具是基于语言模型的机器翻译系统，能够将任何语言的短语翻译成英语。更具体地说，我们使用600M参数的NLLB（Costa-jussà 等人，2022）作为我们的多语言机器翻译模型，支持200种语言（包括低资源语言）。源语言通过`fastText`分类器（Joulin 等人，2016）自动检测，而目标语言始终设定为英语。

Calendar 我们的最终工具是一个日历API，调用时会返回当前日期，无需任何输入。这为需要时间感知的预测提供了时间背景。

4 实验

我们探讨了我们的方法是否能够使模型在没有任何额外监督的情况下自行学习使用工具，并自主决定何时以及如何调用可用的工具。为了验证这一点，我们选择了多种下游任务，在这些任务中我们假设至少有一种考虑的工具是有用的，并在零样本设置下评估性能（第4.2节）。此外，我们还确保我们的方法不会损害模型的核心语言建模能力；我们通过观察两个语言建模数据集上的困惑度（第4.3节）来验证这一点。最后，我们研究了模型规模对学习使用工具能力的影响（第4.4节）。

4.1 实验设置

数据集生成在所有实验中，我们使用CCNet（Wenzek 等人，2020）的子集作为我们的语言建模数据集 $\mathcal{C}$ 以及GPT-J（Wang 和 Komatsuzaki，2021）作为我们的语言模型 M 。为了降低使用API调用进行标注的计算成本，我们为部分API定义了启发式规则，以获取一部分在这些情况下API调用比普通文本更有帮助的文本子集。比如，只有当文本中包含至少三个数字时，才考虑使用计算器工具。所用启发式规则的详细信息见

API 名称	请提供需要翻译的文本内容。	Toolformer: 语言模型可以自我学习使用工具
问答	骑士们在哪里哥伦布建立的?	纽黑文, 康涅狄格州
维基百科搜索	钓鱼卷线器类型	旋转钓法 > 旋转钓法主要分为飞蝇钓和诱饵钓根据所使用的竿和线轮类型进行钓鱼投掷。线轮有两种类型用于旋转钓法时, 开放式卷线器和闭合式卷线器。
计算器	* 2	35
日历	ϵ	今天是2023年1月30日, 星期一。
机器翻译	核安全	核安全

T表1: 所有API的输入和输出示例使用。

API	示例数量		
	$\tau_f = 0.5$	$\tau_f = 1.0$	$\tau_f = 2.0$
问答	51,987	18,526	5,135
维基百科搜索	207,241	60,974	13,944
计算器	3,680	994	138
日历	61,811	20,587	3,007
机器翻译	3,156	1,034	229

表2: 在不同过滤阈值 τ_f 下, 包含API调用的示例数量 (C^*)

附录 A. 为获得 C^* 从 C , 我们执行第 2 节中描述的所有步骤, 并额外筛选出所有在筛选步骤中被全部API调用剔除的示例。对于加权函数, 我们使用

$$w_t = \frac{\tilde{w}_t}{\sum_{s \in \mathbb{N}} \tilde{w}_s} \text{ with } \tilde{w}_t = \max(0, 1 - 0.2 \cdot t)$$

为了确保 API 调用发生在信息对模型实际有帮助的地方附近, 阈值 τ_s 和 τ_f 为每个工具单独选择, 以确保示例数量足够; 详见附录A中的详细信息。表2展示了我们最终数据集(加入 API 调用后)的相关统计数据。

模型微调Wefinetune在上 C^* , 采用批量大小为128, 学习率为 $1 \cdot 10^{-5}$, 在训练的前10%进行线性预热。我们的微调过程的详细信息见附录B。

基线模型在本节的其余部分, 我们主要比较以下模型:

虽然这种过滤会改变训练样本的分布, 但我们假设剩余的样本与原始分布足够接近, 以至于模型的语言建模能力保持不受影响。该假设在第4.3节中得到了实证验证。

• **GPT-J**: 未经过任何微调的常规GPT-J模型。

• **GPT-J +CC**: 在 C 上微调的GPT-J, 我们的CCNet子集没有任何API调用。

Toolformer: 在 C^* 基础上微调的GPT-J, 我们的子集为CCNet, 并增强了API调用。

Toolformer (已禁用): 与Toolformer相同的模型, 但在解码过程中API调用被禁用。

对于大多数任务, 我们还将其与OPT (66B) (Zhang等, 2022) 和GPT-3⁶ (175B) (Brown等, 2020) 进行比较, 这两种模型的规模分别比我们的其他基线模型大约10倍和25倍。

4.2 下游任务

我们在各种下游任务上评估所有模型。在所有情况下, 我们都采用提示的零样本设置——即, 模型被指示用自然语言解决每个任务, 但不提供任何上下文中的示例。这与之之前关于工具使用的研究(例如, Gao 等人, 2022; Parisi 等人, 2022) 不同, 后者会为模型提供特定数据集的示例, 说明如何使用工具来解决具体任务。我们选择更具挑战性的零样本设置, 因为我们希望观察Toolformer是否在用户未提前指定使用哪些工具以及以何种方式解决特定问题的情况下依然有效。

我们使用标准的贪婪解码, 但对Toolformer进行了一个修改: 我们让模型在 $\langle \text{API} \rangle$ 是最可能的情况下开始进行API调用, 而不仅仅是当它是最可能的情况

这是通过手动将 $\langle \text{API} \rangle$ 标记的概率设为0来实现的。我们使用的是未在任何指令上进行微调的原始davinci 变体。

Toolformer: 语言模型可以自我学习使用工具

4.2.1 LAMA

我们在LAMA基准的SQuAD、Google-RE和T-REx子集上评估我们的模型（Petroni等人，2019）。对于这些子集中的每一个任务，都是用缺失事实（例如日期或地点）来完成一段简短的陈述。由于LAMA最初设计用于评估掩码语言模型（例如Devlin等人，2019），我们会过滤掉掩码标记不是最后一个标记的示例，以便剩余的示例可以按从左到右的方式进行处理。为了考虑不同的分词方式以及未通知模型需要单个词的额外复杂性，我们采用比完全匹配更宽松的评估标准，简单地检查正确的词是否在模型预测的前五个词之内。由于LAMA基于直接从维基百科获取的陈述，我们阻止Toolformer使用维基百科搜索API，以避免其获得不公平的优势。

所有模型的结果见表格Results for all models can be seen in Table3. 所有不使用工具的GPT-J模型表现相似。关键是，Toolformer明显优于这些基线模型，分别在最佳基线基础上提升了11.7、5.2和18.6分。它还明显优于OPT（66B）和GPT-3（175B），尽管这两个模型规模更大。这是因为模型几乎在所有情况下（98.1%）自主决定向问答工具请求所需信息；仅在极少数示例中（0.7%）使用了不同的工具，或者（1.2%）没有使用任何工具。<style>

4.2.2 数学数据集

我们在ASDiv（Miao等人，2020）、SVAMP（Patel等人，2021）以及MAWPS基准（Koncel-Kedziorski等人，2016）上测试了数学推理能力。鉴于我们在零样本设置下对所有模型进行测试，我们采用了更宽松的评估标准：由于所需输出始终为数字，我们只需检查第一个结果。

模型	SQuAD Google-RE T-REx		
GPT-J	17.8	4.9	31.9
GPT-J + CC	19.2	5.6	33.2
Toolformer（已禁用）	22.1	6.3	34.9
Toolformer	33.8	11.5	53.5
OPT (66B)	21.6	2.9	30.1
GPT-3 (175B)	26.8	7.0	39.8

表3: LAMA子集的结果。Toolformer在大多数示例中使用问答工具，明显优于所有同规模的基线，并取得与GPT-3（175B）相竞争的结果。

模型	ASDiv	SVAMP	MAWPS
GPT-J	7.5	5.2	9.9
GPT-J + CC	9.6	5.0	9.3
Toolformer（已禁用）	14.8	6.3	15.0
Toolformer	40.4	29.4	44.0
OPT (66B)	6.0	4.9	7.9
GPT-3 (175B)	14.0	10.0	19.8

表4: 需要数学推理的各种基准测试结果。Toolformer在大多数示例中使用计算器工具，明显优于OPT（66B）和GPT-3（175B）。

模型预测的数字。⁷

表4 显示所有基准测试的结果。虽然GPT-J 和 GPT-J CC 的表现大致相当，但Toolformer 在禁用 API 调用的情况下仍取得了更优的结果。我们推测这是因为模型在大量API 调用及其结果的示例上进行了微调，从而提升了其自身的数学能力。尽管如此，允许模型进行 API 调用可以使所有任务的性能提升超过一倍，并且明显优于规模更大的 OPT 和 GPT-3 模型。这是因为在所有基准测试中，97.9%的示例中，模型都会选择请求计算器工具的帮助。

4.2.3 问答系统

我们观察了Web Questions（Berant等人，2013）、Natural Questions（Kwiatkowski等人，2019）和TriviaQA（Joshi等人，2017）这三个由Brown等人（2020）考虑的问答数据集。为了评估，我们检查模型预测的前20个词是否包含正确答案，而不是要求完全匹配。对于Toolformer，我们禁用问答工具，因为

除非模型的预测包含方程式（例如，“正确答案是5+3=8”），否则我们将“=”符号后面的第一个数字视为其预测。

模型	WebQS	NQ	TriviaQA
GPT-J	18.5	12.8	43.9
GPT-J +CC	18.4	12.2	45.6
Toolformer（已禁用）	18.9	12.6	46.7
Toolformer	26.3	17.7	48.8
OPT (66B)	18.6	11.4	45.7
GPT-3 (175B)	<u>29.0</u>	<u>22.6</u>	<u>65.9</u>

表5：各种问答数据集的结果。大多数示例使用维基百科搜索工具，Toolformer明显优于同规模的基线模型，但仍不及GPT-3（175B）。

这将使解决任务变得轻而易举，尤其考虑到基础的问答系统是基于Natural Questions进行微调的。

结果显示在表中5。再次，Toolformer在所有基于GPT-J的模型中表现最优，这次主要依赖于维基百科搜索API（99.3%）来查找相关信息。然而，Toolformer仍然落后于规模更大的GPT-3（175B）模型。这可能是由于我们的搜索引擎相对简单（在许多情况下，它返回的结果明显不匹配特定查询）以及Toolformer无法与之交互，例如，通过重新构造查询以获得更有用的结果或浏览多个顶部结果。我们相信，增加此功能是未来研究的一个令人兴奋的方向。

4.2.4 多语言问答

我们在MLQA（Lewis等人，2019）上评估了Toolformer及所有基线模型，MLQA是一个多语言问答基准测试。每个问题都配有一段英文的上下文段落，而问题可以用阿拉伯语、德语、西班牙语、印地语、越南语或简体中文提出。为了完成任务，模型需要理解段落和问题，因此可能会从将问题翻译成英文中受益。我们的评估指标是模型生成的答案（最多10个词）中包含正确答案的比例。

结果显示在表格6中。使用API调用在所有语言中都能持续提升Toolformer的性能，表明它已学会利用机器翻译工具。根据不同的语言，这一工具在所有示例中的使用比例为63.8%至94.9%；唯一的例外是印地语，其机器翻译工具的使用率仅为7.3%。然而，Tool-

模型	Es	De	Hi	Toolformer: 语言模型 可以自监督学习使用工具		Ar
GPT-J	15.2	16.5	1.3	8.2	18.2	8.2
GPT-J + CC	15.7	14.9	0.5	8.3	13.7	4.6
Toolformer（已禁用）	19.8	11.9	1.2	10.1	15.0	3.1
Toolformer	<u>20.6</u>	13.5	1.4	<u>10.6</u>	16.8	3.7
OPT (66B)	0.3	0.1	1.1	0.2	0.7	0.1
GPT-3 (175B)	3.4	1.1	0.1	1.7	17.7	0.1
GPT-J（全英语）	24.3	27.0	23.9	23.3	23.1	23.6
GPT-3（All En）	24.7	27.2	26.1	24.9	23.6	24.0

表6：在MLQA上针对西班牙语（Es）、德语（De）、印地语（Hi）、越南语（Vi）、中文（Zh）和阿拉伯语（Ar）的结果。虽然使用机器翻译工具进行问题翻译在所有语言中都具有帮助，但在CCNet上进行进一步预训练会降低性能；因此，Toolformer并未在所有情况下持续优于GPT-J。最后两行对应的是提供英文上下文和问题的模型。

Toolformer并不始终优于基础的GPT-J。这主要是因为对于某些语言，针对CCNet的微调会导致性能下降；这可能是由于与GPT-J原始预训练数据相比存在分布偏移。

OPT 和 GPT-3 在所有语言中的表现都出乎意料地较弱，主要原因是它们未能在被指示用英语回答时提供答案。GPT-J 不受此问题困扰的潜在原因是它在训练时使用了比 OPT 和 GPT-3 更丰富的多语种数据，包括 EuroParl 语料库（Koehn, 2005；Gao 等，2020）。作为上限，我们还在 MLQA 的一个变体上评估了 GPT-J 和 GPT-3，其中上下文和问题均以英语提供。在此设置中，GPT-3 的表现优于所有其他模型，支持我们的假设，即其在 MLQA 上表现不佳是由于任务的多语种特性。

4.2.5 时序数据集

为了评估日历API的实用性，我们对所有模型在T<EMP>LAMA（Dhingra等人，2022）和我们新提出的数据集D<ATESET>上进行了评估。T<EMP>LAMA是一个基于维基数据构建的数据集，包含关于随时间变化的事实的完形填空查询（例如，“Cristiano Ronaldo效力于< v23>”），以及2010年至2020年间的正确答案。D<ATESET>，在附录D中描述，也通过一系列模板生成，但采用随机日期/持续时间的组合进行填充（例如，“30天前是星期几？”）。关键的是，回答这些问题需要知道当前日期。

模型	TEMPLAMA	DATASET
GPT-J	13.7	3.9
GPT-J + CC	12.9	2.9
Toolformer（已禁用）	12.7	5.9
Toolformer	16.3	27.3
OPT (66B)	14.5	1.3
GPT-3 (175B)	15.5	0.8

表7：时间数据集的结果。Toolformer优于所有基线，但未在TEMPLAMA中使用日历工具。

对于这两个任务，我们采用与原始 LAMAdataset 相同的评估方法。结果显示在表7 表明，Toolformer 在 TEM-PLAMA 和 DATASET 上的表现均优于所有基线。然而，进一步观察发现，TEMPLAMA 上的改进不能归因于日历工具，该工具仅在所有示例中使用了 0.2%，主要归因于维基百科搜索和问答工具，Toolformer 调用次数最多。这是合理的，因为 TEMPLAMA 中的命名实体通常非常具体且罕见，即使知道确切日期也帮助不大。对于该数据集的最佳处理方式——首先查询日历 API 以获取当前日期，然后用该日期查询问答系统——不仅被我们每个示例最多调用一次 API 的限制所禁止，而且考虑到训练数据中的所有 API 调用都是独立采样的，这对于 Toolformer 来说也难以学习。

对于 DATASET，另一方面，Toolformer 相较于其他模型的显著提升可以完全归因于日历工具，它在所有示例中使用了 54.8%。

4.3 语言建模

除了验证在各种下游任务中性能的提升外，我们还希望确保通过 API 调用进行微调后，Toolformer 的语言建模性能不发生退化。为此，我们在两个语言建模数据集上对模型进行了评估：WikiText (Merity 等人, 2017) 和从 CCNet 中随机抽取的 10,000 篇文档子集 (Wenzek 等人, 2020)，这些文档在训练过程中未被使用。各种模型的困惑度如表8所示。正如预期，在 CCNet 上的微调略微提升了在另一个 CCNet 子集上的性能，但也略微降低了在 WikiText 上的表现，可能是因为原始预训练模型在这两个数据集上的差异。

模型	WikiText	CCNet
GPT-J	9.9	10.6
GPT-J + CC	10.3	10.5
Toolformer（已禁用）	10.3	10.5

表8：不同模型在 WikiText 及我们 CCNet 验证子集上的困惑度。添加 API 调用在不使用 API 调用的情况下，对语言模型的困惑度没有影响。

GPT-J 的训练数据与 Wiki-Text 的相似度高于我们随机选择的 CCNet 子集。然而，最重要的是，在 C^* （我们带有 API 调用标注的数据集）上进行训练，并不会导致困惑度的增加，相较于在推理时禁用 API 调用的情况下在 C 上训练。⁸

4.4 规模定律

我们研究了在调整语言模型 (LM) 规模时，向外部工具寻求帮助的能力如何影响性能。为此，我们不仅将该方法应用于 GPT-J，还应用于来自 GPT-2 系列的四个较小模型 (Radford 等人, 2019)，参数分别为 124M、355M、775M 和 16 亿。我们仅使用三个工具的子集：问答系统、计算器和维基百科搜索引擎。除此之外，我们遵循第 4.1 节中描述的实验设置。

<Figure> 显示，利用所提供工具的能力仅在大约 775M 参数时才开始出现：较小的模型在有无工具的情况下都能实现类似的性能。一个例外是主要用于问答基准的维基百科搜索引擎；我们假设这是因为该 API 相对容易使用。随着模型规模的扩大，模型在解决任务时变得更擅长于 <API 调用>，同时其有效利用所提供 API 的能力也在提升。因此，即使对于我们最大的模型，带有 API 调用和不带 API 调用的预测之间仍存在较大差距。

5 分析

解码策略 我们研究了在第 4.2 节中引入的改进解码策略的效果

⁸ 我们不评估启用 API 调用的 Toolformer 的困惑度，因为计算给定 x_t 的概率 $p_M(x_t | x_1 \dots x_{t-1})$ 需要对模型在位置 t 可能进行的所有潜在 API 调用进行边际化，这在计算上是不可行的。

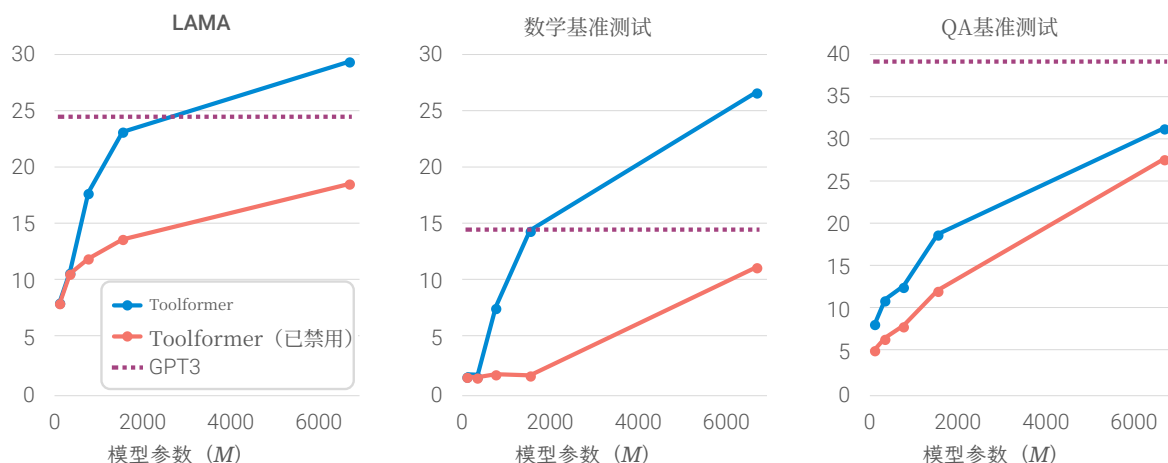


图4: 不同规模GPT-2模型及采用我们方法微调的GPT-J在LAMA、我们的数学基准和问答基准上的平均表现, 包含有无API调用的情况。虽然API调用对最小模型帮助不大, 但较大模型学会了如何有效利用它们。即使对于更大的模型, 带有和不带有API调用的模型预测之间的差距仍然很大。

most likely token, we generate the `<API>` token if it is one of the k most likely tokens. Table 9 显示了在不同 k 值下, T-REx 子集的性能表现以及 WebQS 的表现。如预期所示, 增加 $k=1$ 会使模型对更多示例进行 API 调用——从 (即常规贪婪解码) 达到 40.3% 和 8.5%, 到 k 为 98.1% 和 100%。在 T-REx 上, 贪婪解码已经明显提升了性能, 而在 WebQS 上, 模型仅在略微增加 $k=1$ 时开始进行大量 API 调用。有趣的是, 对于 {v34}, 模型在一定程度上是校准的: 它会选择对某些示例调用 API, 即使没有调用 API, 其表现也会特别差。这可以从模型在决定 `<style id='38'>` 不调用 API 的示例 (44.3 和 19.9) 上的表现高于完全不调用 API 时的平均水平 (34.9 和 18.9) 看出。然而, 对于更高的。

数据质量 我们对使用我们的方法生成的不同 API 调用进行了定性分析。表 10 展示了来自 CCNet 的文本示例, 这些示例经过 API 调用增强, 以及相应的得分 $L_i^- - L_i^+$, 该得分用作筛选标准, 以及模型所做的 API 调用在给定上下文中是否直观上有帮助。如图所示, 高值的 $L_i^- - L_i^+$ 通常对应有用的 API 调用, 而低值则对应未提供有助于预测未来标记信息的 API 调用。也存在一些例外, 例如

k	T-REx				WebQS			
	所有交流	NC	%		所有交流	NC	%	
0	34.9	—	34.9	0.0	18.9	—	18.9	0.0
1	47.8	53.0	44.3	40.3	19.3	17.1	19.9	8.5
3	52.9	58.0	29.0	82.8	26.3	26.5	6.6	99.3
10	53.5	54.0	22.5	98.1	26.3	26.4	—	100.0

表 9: Toolformer 在 LAMA 的 T-REx 子集和 WebQS 上在不同 k 取值下的解码表现。显示的数字包括整体性能 (All)、模型决定调用 API 的子集上的性能 (AC) 以及其余示例 (NC), 以及模型决定调用 API 的示例所占的百分比 (%)。

“快速列车成功”在第四个示例中没有提供任何相关信息, 但仍然降低了困惑度。然而, 未经过滤的 API 调用中存在一定程度的噪声实际上是有用的, 因为它迫使在 C^* 上微调的模型不总是盲目地遵循每次调用的结果。

6 相关工作

Language Model Pretraining 存在多种方法在预训练过程中增强语言模型, 加入某种形式的额外文本信息, 包括各种形式的元数据 (Keskar et al., 2019)、HTML 标签 (Aghajanyan et al., 2021)、维基百科标记 (Schick et al., 2022), 或从信息检索系统获取的相关文本 (Gua et al., 2020; Borgeaud et al., 2021; Izacard et al., 2022)。对于所有这些

示例	$L_i^- - L_i^+$	有用
弗洛登窗（纪念米德尔顿弓箭手的战争纪念碑），位于一级保护等级的米德尔顿圣伦纳德教堂被认为是英国最古老的战争纪念碑之一 dom. <code><API> WikiSearch(弗洛登战役纪念馆) → 弗洛登战役 >纪念活动 > Middleton Parish Church</code> 中的彩色玻璃弗洛登窗 [。 . .]由Sir建造 理查德·阿谢顿纪念弗洛登战役及来自米德尔顿的弓箭手们 <code></API></code> 米德尔顿的理查德·阿谢顿爵士（建造了圣伦纳德教堂）被授予骑士身份 [。 . .]	5.49	✓
注意： WL 将于星期五开放， <code><API> 日历()</code> → 今天是2017年3月9日，星期四。 <code></API></code> 3月10日，以及3月19日星期日，正常营业时间。	2.11	✓
尼罗河 的长度大约为 <code><公里API> QA</code> （尼罗河的大致长度是多少？） ,853 公里 <code></API></code> 6，白尼罗河为其主要源头。	2.08	✓
如果金星拥有类似地球的大气层，那么你会预期金星的平均温度为 499 K (1.74×287) 而不是735 K，即 <code><API> Calculator(735 / 499)</code> → 1.47 <code></API></code> 1.47 (735 / 499) 倍于应有的温度。	1.59	✓
你在这里：主页 / 精选 / 抓住这班快速列车，迈向成功！ <code><API> WikiSearch(快速列车成功)</code> → 快速训练 > 它在加拿大CHUM单曲榜上也达到了第23名，6月1971年26月。这一单曲的成功确立了Myles Goodwyn作为乐队主要词曲作者的地位。 并使得 April Wine 得以录制第二张专辑。 <code></API></code> 不要等待数周和 [。 . .]	0.92	✗
泽西岛的最佳学校 2020 <code><API> MT(泽西岛的最佳学校)</code> → 最佳泽西岛的学校 <code></API></code> 在本页面，您可以搜索大学、学院和商学院 泽西岛的学校	0.70	✓
享受这些来自 <code>API> Calendar()</code> → 今天是2013年4月19日，星期五。 <code></API></code> 彩蛋寻宝	0.33	✓
85名患者（23%）在生存状态下住院，并被送入医院病房。其中， <code><API> Calcula-</code> → 3.70 <code></API></code> 65% 具有心源性病因 [。 . .]	-0.02	✗
但是嘿，在 <code><API> Calendar()</code> → 今天是2011年6月25日，星期六。 <code></API></code> 迪士尼乐园 在消防演习中出现的失误，我认为可以放心地说，Chewey 不会让任何人在火灾中丧生。	-0.41	✗
上次我与 <code><API> QA</code> （上次我与谁在一起？） → 上次 <code></API></code> 我问他喜欢我什么，他说有一天会告诉我。	-1.23	✗

表10：不同工具的API调用示例，按 $L_i^- - L_i^+$ 的值排序，作为过滤标准。较高的值通常对应于在预测未来标记时直观有用的API调用。

方法，无论是否有帮助，都会提供额外的信息始终。相比之下，Toolformer 会自主学习，明确地请求所需的正确信息。

工具使用几种方法旨在赋予语言模型使用外部工具的能力，例如搜索引擎（Komeili et al., 2022; Thoppilan et al., 2022; Lazaridou et al., 2022; Shuster et al., 2022; Yao et al., 2022）、网页浏览器（Nakano et al., 2021）、计算器（Cobbe et al., 2021; Thoppilan et al., 2022）、翻译系统（Thoppilan et al., 2022）以及Python解释器（Gao et al., 2022）。这些模型学习使用工具的方法大致可以分为两类：要么依赖大量的人类监督（Komeili et al., 2022; Nakano et al., 2021; Thoppilan et al., 2022），要么通过在针对特定任务的少样本提示设置中引导语言模型，使其提前知道需要使用哪些工具。

used (Gao et al., 2022; Lazaridou et al., 2022; Yao et al., 2022). 相较之下，Toolformer的自监督特性使其能够学习何时以及如何使用工具，而无需提供显示任务特定示例的提示，展示工具的使用方式。或许与我们的工作最为相关的是TALM（Parisi et al., 2022），这是一种采用类似自监督目标的方法，用于教导模型使用计算器和搜索引擎，但仅在模型为下游任务进行微调的设置中进行探索。

自举利用自训练和自举技术以提升模型性能的想法已在多种语境中得到研究，涵盖词义消歧（Yarowsky, 1995）、关系抽取（Brin, 1999; Agichtein 和 Gravano, 2000）、句法分析（McClosky 等人, 2006; Reichart 和 Rappoport, 2007）、序列生成（He 等人, 2020）、少样本文本分类—

fication (Schick and Schütze, 2021a) 和检索 (Izacard and Grave, 2021) 以进行推理 (Zelikman et al., 2022)。与这些方法类似, Toolformer 在应用基于困惑度的筛选步骤后, 利用其自身的预测进行训练。

7 限制

虽然我们的方法使得语言模型能够以自监督的方式学习如何使用各种工具, 但在目前的形式下, 我们的方法在实现方面仍存在一些明显的局限性。其中一个限制是Toolformer无法进行链式(即将一个工具的输出作为另一个工具的输入)使用。这是因为每个工具的API调用是独立生成的; 因此, 在微调数据集中没有链式工具使用的示例。我们当前的方法也不允许语言模型以交互式的方式使用工具——尤其是对于搜索引擎等可能返回数百个不同结果的工具, 这样的工具可以让模型浏览这些结果或细化搜索查询, 类似于Nakano等人(2021)的做法, 这对于某些应用可能至关重要。此外, 我们发现用Toolformer训练的模型在决定是否调用API时, 往往对输入的措辞非常敏感; 这或许不足为奇, 因为已知语言模型在零样本和少样本设置中对提供的提示非常敏感 (Jiang等人, 2020; Schick和Schütze, 2021a)。根据工具的不同, 我们的方法在样本效率方面也存在很大问题; 例如, 处理超过一百万份文档, 仅能获得几千个有用的调用计算器API的示例。解决这一问题的潜在方案可能是迭代应用我们的方法, 类似于相关引导方法中的做法 (Schick和Schütze, 2021a; Izacard和Grave, 2021; Parisi等人, 2022)。最后, 在决定是否调用API时, Toolformer目前尚未考虑调用API所带来的工具相关的计算成本。

8 结论

我们介绍了Toolformer, 一种通过自监督学习方式, 学习如何使用搜索引擎、计算器和翻译系统等不同工具的语言模型。这一过程通过在大量采样的API调用上进行微调实现, 这些API调用会根据其是否满足特定条件进行筛选。

降低未来词元的困惑度。Toolformer显著提升了6.7B参数GPT-J模型的零-shot性能, 使其甚至在一系列不同的下游任务中超越了规模更大的GPT-3模型。

参考文献

- Armen Aghajanyan, Dmytro Okhonko, Mike Lewis, Mandar Joshi, Hu Xu, Gargi Ghosh, 和 Luke Zettlemoyer. 2021.
- Eugene Agichtein 和 Luis Gravano. 2000. Snowball: 从大量纯文本集合中提取关系。在第五届 ACM 数字图书馆会议论文集, DL '00, 第 85–94 页, 纽约, 美国。计算机协会。
- Ricardo Baeza-Yates, Berthier Ribeiro-Neto 等人, 1999年。现代信息检索, 第463卷。ACM出版社, 纽约。
- Jonathan Berant, Andrew Chou, Roy Frostig, 和 Percy Liang. 2013. 通过问答对在 Freebase 上进行语义解析。收录于《2013 年自然语言处理实证方法会议论文集》, 第1533–1544页, 华盛顿州西雅图, 美国计算语言学会。
- 通过检索数万亿个标记来提升语言模型的性能
- Sergey Brin. 1999. 从万维网中提取模式与关系。在《万维网与数据库》, 第 172–183 页, 柏林、海德堡。Springer Berlin Heidelberg。
- 语言模型是少样本学习者

Toolformer: 语言模型如何自我学习使用工具

Toolformer: 语言模型如何自我学习使用工具

Gautier Izacard 和 Edouard Grave. 2021. 从阅读器到检索器的知识蒸馏用于问答 在国际学习表征会议。

Atlas: 结合检索增强的少样本学习语言模型

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano 等人. 2021. 训练验证器以解决数学应用题. *arXiv preprint arXiv:2110.14168*.

Marta R Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard 等. 2022. 无语言被遗忘: 规模化以人为本的机器翻译. *arXiv preprint arXiv:2207.04672*.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, 和 Kristina Toutanova. 2019年。

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, 和 Pascale Fung. 2022.

姜正宝, 徐方芳, 荒木淳, Neubig Graham. 2020.

Mandar Joshi, Eunsol Choi, Daniel Weld, 和 Luke Zettlemoyer. 2017年。

Bhuwan Dhingra, Jeremy R. Cole, Julian Martin Eisenschlos, Daniel Gillick, Jacob Eisenstein, 和 William W. Cohen. 2022.

Gao Leo, Biderman Stella, Black Sid, Gold- ing Laurence, Hoppe Travis, Foster Charles, Phang Jason, He Horace, Thite Anish, Nabeshima Noa 等人. 2020. The pile: 一份包含多样文本的800GB数据集, 用于语言建模. *arXiv preprint arXiv:2101.00027*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, 和 Graham Neubig. 2022.

Armand Joulin, Edouard Grave, Piotr Bojanowski, Matthijs Douze, Herve Jégou, 和 Tomas Mikolov. 2016. Fasttext.zip: 压缩文本分类模型. *arXiv 预印本 arXiv:1612.03651*.

Nitish Shirish Keskar, Bryan McCann, Lav R. Varshney, Caiming Xiong, 和 Richard Socher. 2019.

Philipp Koehn. 2005. Europarl: 用于统计机器翻译的平行语料库. 收录于机器翻译峰会X: 论文集, 第79–86页。

Mojtaba Komeili, Kurt Shuster, 和 Jason Weston. 2022. 互联网增强对话生成. 收录于第60届计算语言学协会年会 (第1卷: 长论文) 论文集, 页码8460–8478, 爱尔兰都柏林. 计算语言学协会。

Rik Koncel-Kedzior, Subhro Roy, Aida Amini, Nate Kushman, 和 Hannaneh Hajishirzi. 2016年。MAWPS: 数学应用题库在 2016年北美计算语言学会人类语言技术会议论文集: 人类语言技术, 第1152-1157页, 圣地亚哥, 加利福尼亚。计算语言学会。

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. 自然问题: 问答研究的基准 《计算语言学协会交易》, 7:452–466.

Angeliki Lazaridou, Elena Gribovskaya, Wojciech Stokowiec, 和 Nikolai Grigorev. 2022. 通过少样本提示实现互联网增强的语言模型用于开放域问答. *arXiv preprint arXiv:2203.05115*.

Patrick Lewis, Barlas Oğuz, Ruty Rinott, Sebastian Riedel, 和 Holger Schwenk. 2019. Mlqa: 跨语言抽取式问答评估. *arXiv preprint arXiv:1910.07475*.

少样本学习与多语言模型

Joshua Maynez, Shashi Narayan, Bernd Bohnet, 和 Ryan McDonald. 2020.

David McClosky, Eugene Charniak, and Mark Johnson. 2006. 有效的自我训练用于句法分析. 收录于NAACL人类语言技术会议主会议论文集, 第152–159页, 美国纽约市。计算语言学协会。

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). In *International Conference on Learning Representations*.

沈云淼、梁超春、苏克义。2020年。

Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders,

徐江、卡尔·科布、蒂娜·埃隆杜、格雷琴·克鲁格、凯文·巴顿、马修·奈特、本杰明·切斯、约翰·舒尔曼。2021年。

Aaron Parisi, Yao Zhao, 和 Noah Fiedel. 2022.

Arkil Patel, Satwik Bhattamishra 和 Navin Goyal. 2021年。

Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel 和 Sebastian Riedel. 2021 年。

语言模型作为知识库?

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever 等. 2019. 语言模型是无监督的多任务学习者. *OpenAI 博客*, 1(8):9.

Roi Reichart 和 Ari Rappoport. 2007. 基于自训练的统计句法分析器增强与领域适应方法, 训练于小型数据集 发表于第45届计算语言学协会年会议论文集, 页码 616–623, 捷克布拉格。计算语言学协会。

Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford 等. 1995. Okapi 在 TREC-3 中的表现. *Nist Special Publication Sp*, 109:109.

Peer: 一个协作式语言模型

Timo Schick 和 Hinrich Schütze. 2021a. 利用完形填空问题进行少样本文本分类和自然语言推断。

第16届欧洲计算语言学学会分会会议：主论文集，第255–269页，线上。计算语言学协会。

Timo Schick 和 Hinrich Schütze. 2021b. 预训练语言模型生成数据集。在《2021年自然语言处理实证方法会议论文集》，第6943-6951页，线上及多米尼加共和国蓬塔卡纳。计算语言学协会。

Kurt Shuster, Jing Xu, Mojtaba Komeili, Da Ju, Eric Michael Smith, Stephen Roller, Megan Ung, Moya Chen, Kushal Arora, Joshua Lane, Morteza Behrooz, William Ngan, Spencer Poff, Naman Goyal, Arthur Szlam, Y-Lan Boureau, Melanie Kambadur, 和 Jason Weston. 2022.

Toolformer：语言模型可以自我学习使用工具

Ben Wang 和 Aran Komatsuzaki. 2021. GPT-J-6B：一个拥有60亿参数的自回归语言模型。

Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, 和 Hannaneh Hajishirzi. 2022.

大规模语言模型的突现能力

Toolformer：语言模型可以自我学习使用工具

web crawl data . In Proceedings of the Twelfth Language Resources and Evaluation Conference, pages 4003–4012, Marseille, France. European Language Resources Association.

References

David Yarowsky. 1995. 无监督词义消歧，媲美监督方法。第33届计算语言学协会年会，页码189–196，马萨诸塞州剑桥，美国计算语言学协会。

Eric Zelikman, Yuhuai Wu, Jesse Mu, 和 Noah D. Goodman. 2022.

Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, MoyaChen, Shuohui Chen, ChristopherDewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, 和 Luke Zettlemoyer. 2022年。

A API 细节

在采样和筛选API调用时，默认使用 $\tau_s = 0.05$ 和 $\tau_f = 1.0$ 的值——即我们仅在 $\langle \text{API} \rangle$ 标记的概率至少为5%的位置进行API调用，并且当API调用能将损失降低至少1.0时，我们会保留该调用。我们只保留前 $k = 5$ 的这些位置，并在一段文本中为每个识别出的位置采样最多 $m = 5$ 次API调用。由于下面描述的启发式筛选，我们仅对一小部分 C 生成计算器和机器翻译系统的API调用；为弥补这一点，我们为这些工具设置了 $\tau_s = 0.0$ 、 $k = 20$ 和 $m = 10$ 。由于生成的API调用集仍然相对较小，我们还额外设置了 $\tau_f = 0.5$ 。

A.1 实现

问答系统我们使用Izacard等人（2022年）在自然问答（Kwiatkowski等人，2019年）上微调的Atlas模型作为我们的问答系统。为了创建 C^* ，我们采用了Atlas-large模型，能够高效处理数百万次API调用；在推理阶段，我们使用更大的Atlas-xxl模型。

Calculator 我们的计算器基于一个简单的Python脚本，仅支持“+”、“-”、“*”和“/”这几个运算符。对于语法不正确的等式，不会返回任何结果。为了采样API调用，我们对CCNet的子集应用启发式过滤，只处理以下情况的文档：(i) 在100个标记的窗口内包含至少三个数字，其中一个数字是对另外两个数字进行数学运算的结果，(ii) 包含序列“=”、“等于”、“等于”、“总和”或“平均值”后跟一个数字，或(iii) 至少包含三个数字；对于只符合最后一项标准的文本，我们仅保留1%的随机子集。

Calendar 为了创建我们的数据集 C^* ，我们假设在此类情况下的日历日期应为文档创建的日期。我们通过提取URL中的日期（如果存在）来近似此日期。我们过滤掉无法提取日期的文本，剩下大约18%的文档。

机器翻译在训练和推理过程中，我们使用600 M参数的NLLB（Costa-jussà等人，2022）作为我们的机器翻译（MT）模型。源语言通过fastText分类器（Joulin等人，）自动检测。

2016)，而目标语言始终设定为英语。由于大部分CCNet数据集为英语，我们在生成API调用之前过滤掉仅包含英语文本的部分。更具体地说，我们只保留那些包含非英语文本片段且其前后均为英语文本的段落。我们使用长度为10个标记的文本片段。为了判断中间的文本片段是否为非英语，我们再次使用fastText分类器，置信度大于0.8。我们还会过滤掉仅包含数字或特殊符号的文本片段。这一过滤机制使我们能够更高效地生成数据，集中在可能对机器翻译工具有帮助的区域进行API调用。在生成API调用后，我们还会从训练集中移除那些输入到机器翻译工具的内容出现在API调用之后但之前没有出现的样本。虽然在数据生成过程中，模型可以提前预测以生成API调用，但在推理阶段这是不可能的，因此我们希望避免模型在此类情况下调用API。

A.2 提示

以下列出用于采样每个工具的API调用的提示。

问答 我们使用以下提示来进行问答工具：你的任务是向一段文本中添加对问答API的调用。问题应帮助你获取完成文本所需的信息。你可以通过写“[QA(问题)]”来调用API，其中“问题”是你想要提问的问题。以下是一些API调用的示例：输入：乔·拜登出生于宾夕法尼亚州斯克兰顿。输出：乔·拜登出生于 [QA("乔·拜登出生在哪里？")] 斯克兰顿，[QA("斯克兰顿位于哪个州？")] 宾夕法尼亚州。

可口可乐，或称可乐，是由可口可乐公司生产的一种碳酸软饮料。

可口可乐，或 [QA("可口可乐还被称为什么名字？")] 可乐，是由[QA("谁生产可口可乐？")] 可口可乐公司制造的碳酸软饮料。

Toolformer: 语言模型可以自我学习使用工具

Calculator 我们为计算器使用以下提示：

您的任务是向一段文本中添加对 Calculator API 的调用。

调用应帮助您获取完成文本所需的信息。您可以通过写入 “[Calculator(expression)]” 来调用API，其中 “expression” 是待计算的表达式。以下是一些API调用的示例：输入：下一项的数字是 $18 + 12 \times 3 =$ 54。输出：下一项的数字是 $18 + 12 \times 3 =$ [Calculator($18 + 12 \times 3$)]54。

人口为658,893人。这占全国平均人口5,763,868人的11.4%。

人口为658,893人。这占全国平均水平的11.4%，即 [Calculator($658,893 / 11.4\%$)] 5,763,868人。

共进行了252场晋级赛，进球数为723个（平均每场 [Calculator($723 / 252$)] 2.87个）。这一数字比去年2169个进球少了三倍。

共进行了252场符合条件的比赛，进球数为723个（平均每场 [Calculator($723 / 252$)] 2.87个）。比去年多进了20个球。

我在1994年去了巴黎，并一直待到2011年，因此总共是 [Calculator($2011 - 1994$)] 17 年。

由此，我们有 4×30 分钟= 120 分钟。

Toolformer: 语言模型可以自我学习使用工具

维基百科搜索我们为维基百科搜索工具使用以下提示：

您的任务是完成一段给定的文本。您可以使用Wikipedia搜索API进行信息查询。方法是写成 "[WikiSearch(词条)]"，其中“词条”是您想要查找的搜索词。以下是一些API调用的示例：

加纳国旗上的颜色具有以下含义：红色代表烈士的鲜血，绿色象征森林，金色代表矿产财富。

加纳国旗的颜色具有以下含义：红色代表 WikiSearch("Ghana flag red meaning") 烈士的鲜血，绿色象征森林，金色代表矿产财富。

但在纳米材料生产过程中存在哪些风险？一些

纳米材料可能引起各种类型的肺部损伤。

但在纳米材料生产过程中存在哪些风险？ [WikiSearch("纳米材料生产风险")] 一些纳米材料可能引起各种类型的肺部损伤。

二甲双胍是2型糖尿病和肥胖患者的一线药物。

二甲双胍是2型糖尿病和肥胖患者的一线药物。

Toolformer: 语言模型可以自我学习使用工具

机器翻译我们使用以下提示来进行机器翻译工具：

Toolformer: 语言模型可以自我学习使用工具

他已出版一本书：《被压制者》

在 Morris de Jonge 的《Jeschuah》中，der klassische jüdische Mann [MT(der klassische jüdische Mann)]，描述了一位犹太作家

南京高淳县住房和城乡建设局 城市新区设计 a plane of reference Gaochun is one of seven districts of the provincial capital Nanjing

Toolformer: 语言模型可以自我学习使用工具

Calendar 我们为日历工具使用以下提示：

Your task is to add calls to a Calendar API to a piece of text. The API calls should help you get information required to complete the text. You can call the API by writing "[Calendar()]" Here are some examples of API calls:

今天是今年的第一个星期五。

今天是本年度的第一个 [Calendar()] 星期五。

美国总统是乔·拜登。

今天是星期三。

今天是星期[Calendar()] 。

距离圣诞节还有30天。输出：距离圣诞节还有 [Calendar()] 30天。

商店周末从不营业，所以今天 [Calendar()] 它关闭了。

Toolformer：语言模型可以自我学习使用工具

B Toolformer 训练

我们每个API使用最多25,000个示例。最大序列长度为1,024。有效批次大小为128。所有模型均采用DeepSpeed的ZeRO-3（Rasley等，2020）进行训练。我们使用8块NVIDIA A100 40GB GPU，采用BF16。训练最多2,000步，在每500步时，我们在来自CCNet的包含1,000个示例的小型开发集上评估困惑度（PPL）。我们选择表现最佳的检查点。

C 零样本提示

C.1 LAMA与 TEMPLAMA

对于LAMA和TEMPLAMA，无论输入文本为 $\mathbf{x}$ ，我们都使用以下提示：请完成以下文本，使其在事实基础上正确： $\mathbf{x}$ 。

C.2 数学基准测试

对于所有数学基准测试，给定一个上下文 $\mathbf{x}$ 和一个问题 $\mathbf{q}$ ，我们的提示是： $\mathbf{x} \mathbf{q}$ 答案是。

C.3 问答

对于所有问答数据集，包括DATESET，我们只需在问题前加上“回答以下问题：”。如果问题本身没有以问号结尾，我们会添加一个问号。

C.4 多语言问答

对于MLQA，给定一个上下文 $\mathbf{x}$ 和一个问题 $\mathbf{q}$ ，我们的提示是：你的任务是

模板	Size
还有多少天{前是, 直到} <i>past_date, future_date</i> }?	400
什么 {星期几、几月、几日、月份} year} was it (<i>current_date - past_date</i>){days, 几周、几个月、几年} 前?	800
什么 {星期几、几月、几日、月份} 未来_日期 - 当前_日期 天数? 它是在{过去_日期}的星期几? 未来_日期}?	800
什么 {星期几、几月、几日、月份} year} {是, 曾经} 它 {前天, 是} 昨天、今天、明天、后天}	4,000
什么 {星期几、几月、几日} 今年是{is, was}吗?	1,800
有多少{天、周、月、年} {前} 今年为止, 是否有 <i>holiday</i> ?	1,200
Toolformer: 语言模型可以自我学习使用工具	9,400

表11：用于创建DATESET的模板，其中一个 *current_date* 会被随机选择。对于每个 *current_date*，会生成一个随机的 *past_date* 和 *future_date*，并用以填充每个模板（如相关）。涉及假日的模板中使用了美国的联邦假日（例如感恩节）。

根据以下段落回答问题： $\mathbf{x}$ Now answer the following question in English: $\mathbf{q}$.

D DATESET

DATESET 是通过首先随机选择500个“当前日期”创建的。对于每个当前日期，在四年范围内随机选择另一个相对过去或未来的日期，然后用这两个日期填充表11中的查询模板。
11. 使用第一个模板的此类查询示例为：“2020年8月14日距离现在有多少天？”如果调用，日历工具将返回假定的当前日期（例如，“今天是2020年11月20日，星期日”）。